UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA
TRANSFORMATEC



8.Transport Layer: TCP Congestion Control

Computer Networks



CS4054 - 2026II
PROF.: CWILLIAMS@UTEC.EDU.PE, JGONZALEZ@UTEC.EDU.PE
SRC: HARRIS & HARRIS - DIGITAL DESIGN AND COMPUTER ARCHITECTURE RISC-V

Executive Summary

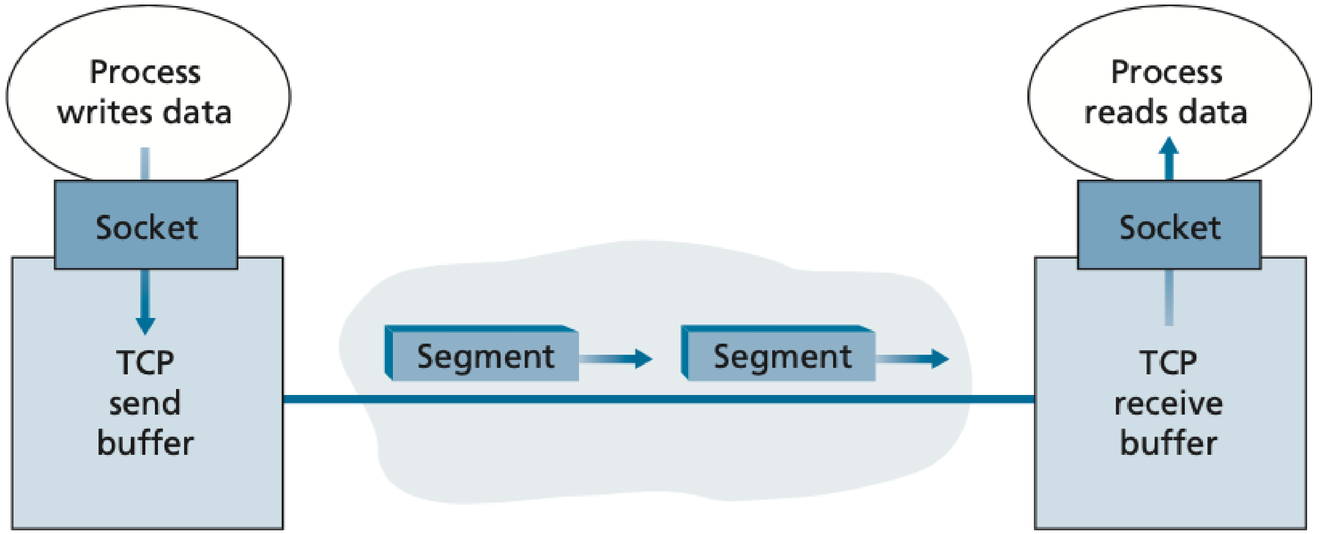
- **Motivation: Network implementation needs control over buffering and congestion.**
- **Problem: How to incorporate buffering and congestion control into a TCP implementation.**
- **Overview:**
  - **Key definitions of network congestion.**
  - **TCP windowing and buffering mechanisms.**
  - **TCP congestion control and fairness.**
- **Conclusion: We extended our understanding of TCP by studying buffering, congestion control, and fairness mechanisms.**

Outline

- Introduction
- Principles of congestion
- TCP Congestion control
- Conclusions

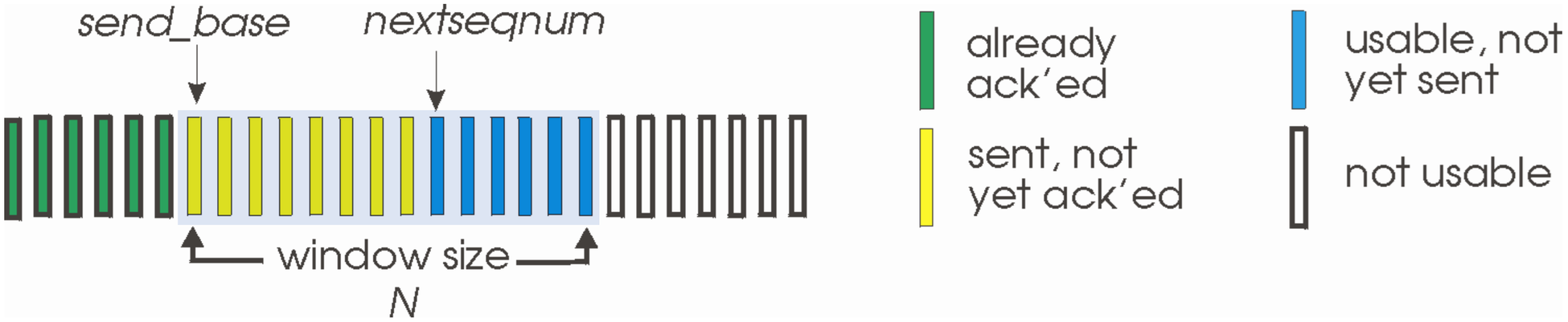
Recall definitions: Window

- **On the receiving system, TCP stores received data in a receive buffer. TCP acknowledges receipt of the data, and *advertises* (communicates) a new *receive window* to the sending system. The receive window represents the number of bytes that are available in the receive buffer.** If the receive buffer is full, the receiving system advertises a receive window size of zero, **and the sending system must wait to send more data.**



Go-Back-N: sender

- sender: “window” of up to N, consecutive transmitted but unACKed pkts
  - k-bit seq # in pkt header

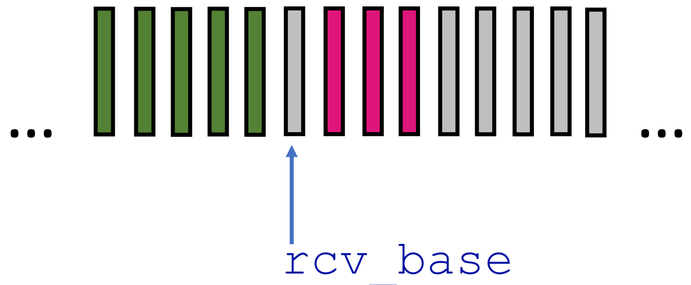


- *cumulative ACK:* ACK(*n*): ACKs all packets up to, including seq # *n*
  - on receiving ACK(*n*): move window forward to begin at *n+1*
- timer for oldest in-flight packet
- *timeout(n):* retransmit packet n and all higher seq # packets in window

Go-Back-N: receiver

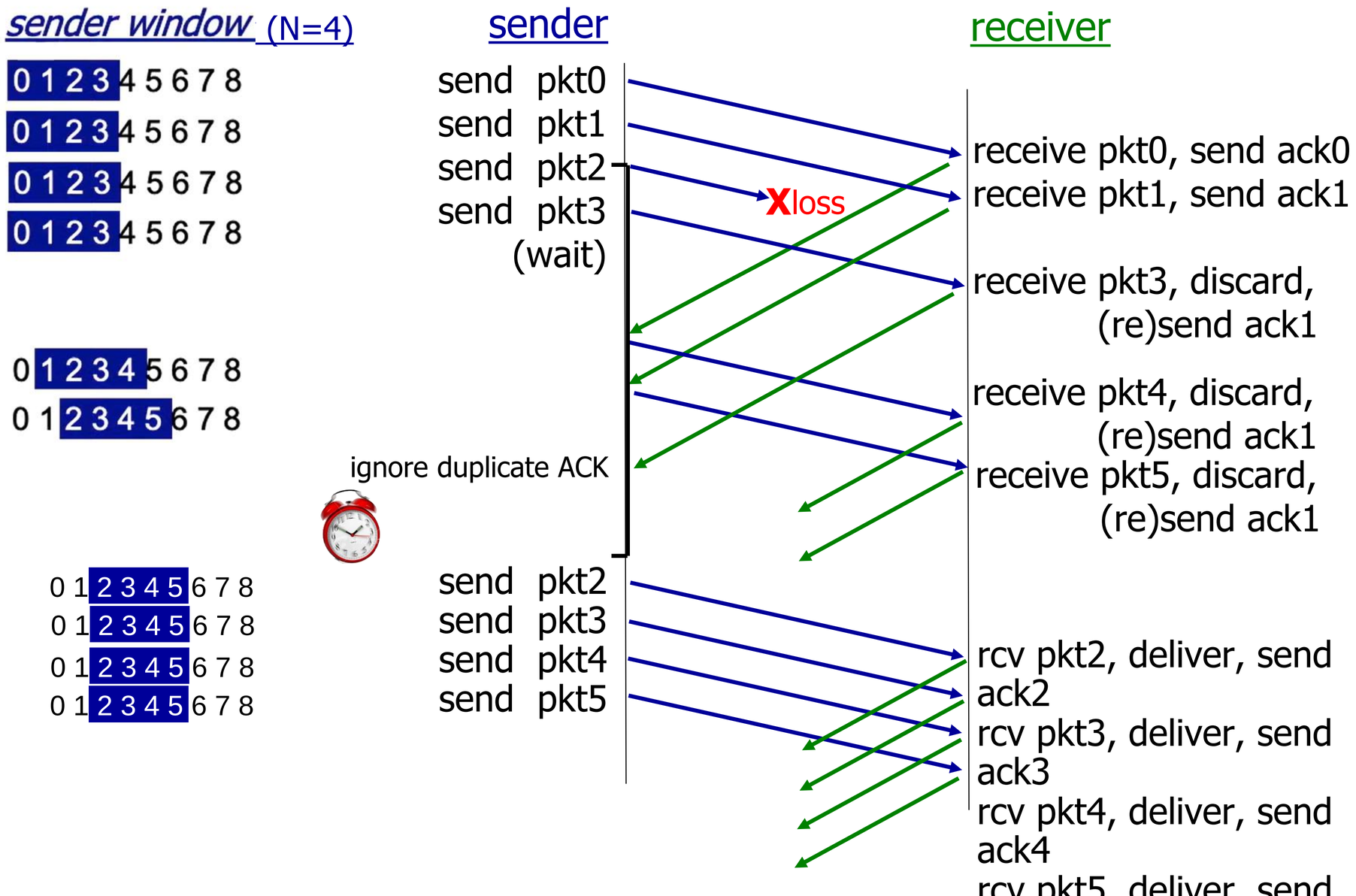
- ACK-only: always send ACK for correctly-received packet so far, with highest *in-order* seq #
  - may generate duplicate ACKs
  - need only remember `rcv_base`
- on receipt of out-of-order packet:
  - can discard (don't buffer) or buffer: an implementation decision
  - re-ACK pkt with highest in-order seq #

Receiver view of sequence number space:





Go-Back-N in action



Outline

- Introduction
- Principles of congestion
- TCP Congestion control
- Conclusions

Principles of congestion control

Congestion:

- informally: “too many sources sending too much data too fast for *network* to handle”
- manifestations:
  - long delays (queueing in router buffers)
  - packet loss (buffer overflow at routers)
- different from flow control!
- a top-10 problem!



congestion control: too many senders, sending too fast





flow control: one sender too fast for one receiver

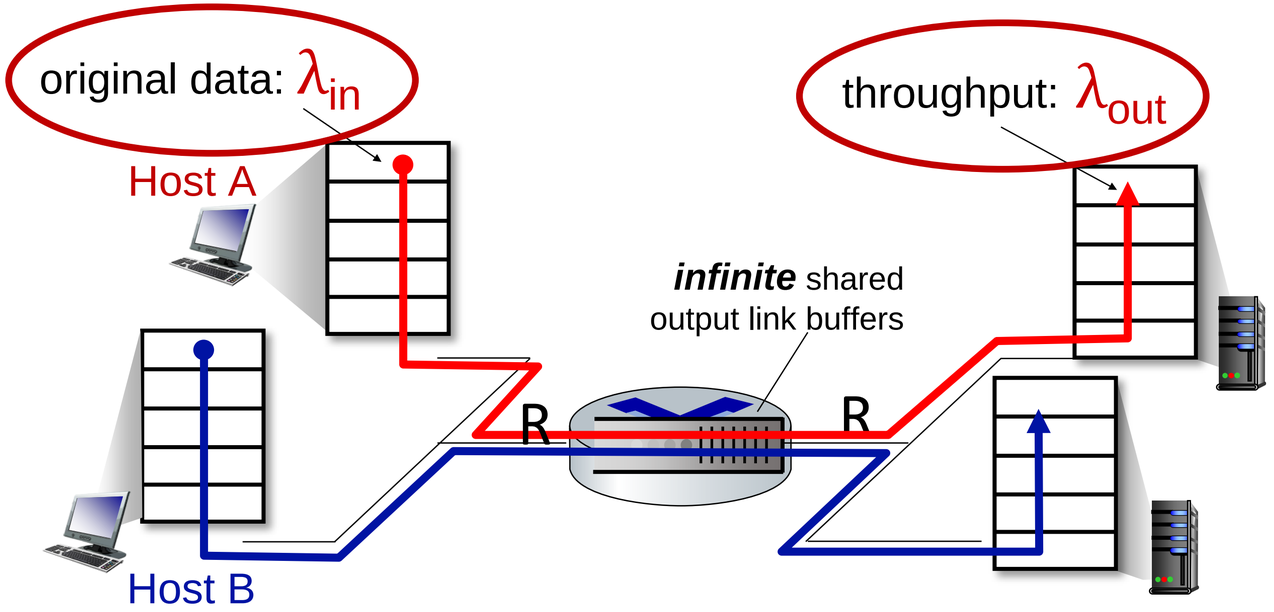
Causes/costs of congestion: scenario 1

Simplest scenario:

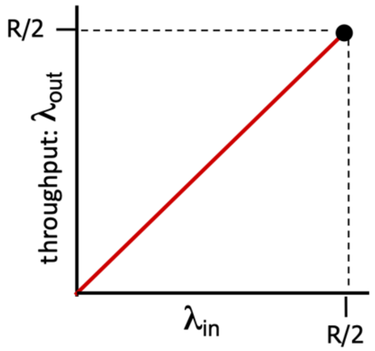
- one router, infinite buffers
- input, output link capacity: R

two flows

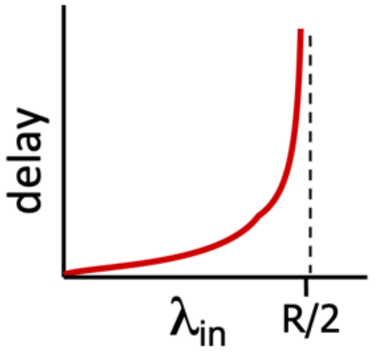
- no retransmissions needed



Q: What happens as arrival rate λ_{in} approaches R/2?



maximum per-connection throughput: R/2

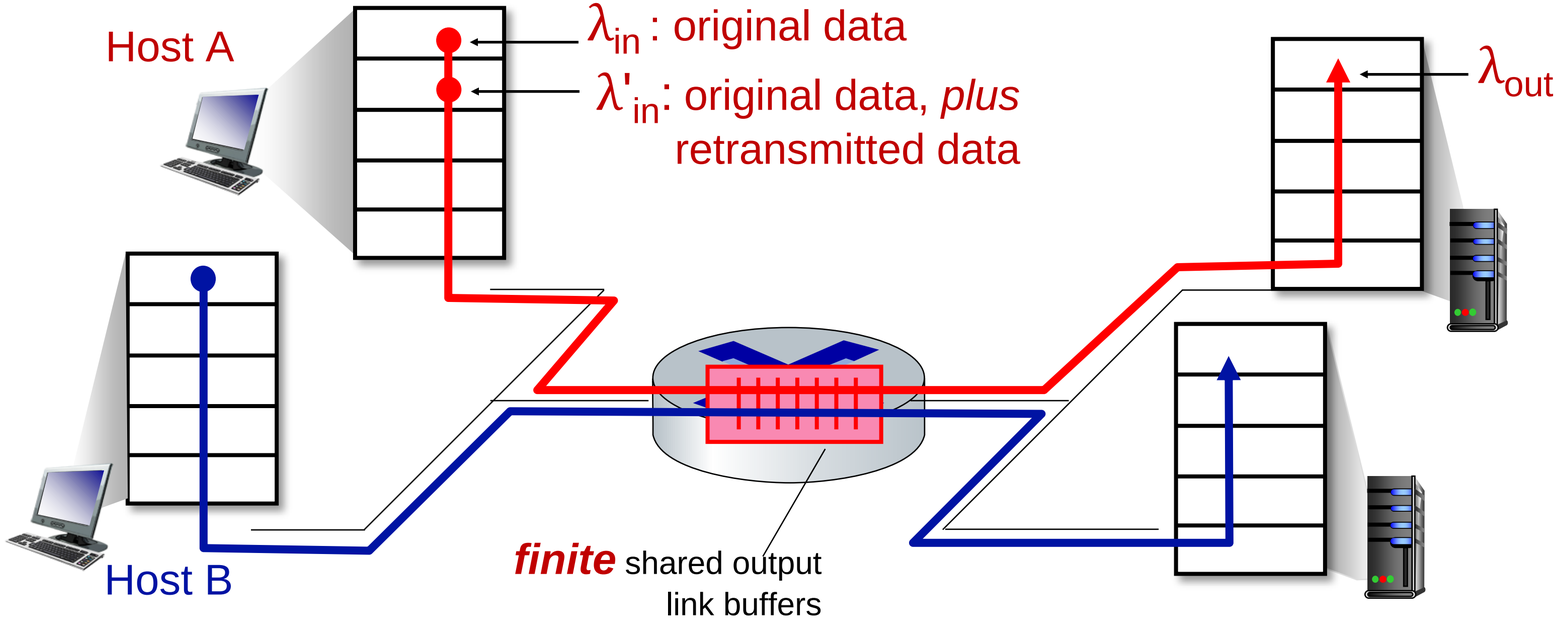


large delays as arrival rate λ_{in} approaches capacity

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Causes/costs of congestion: scenario 2

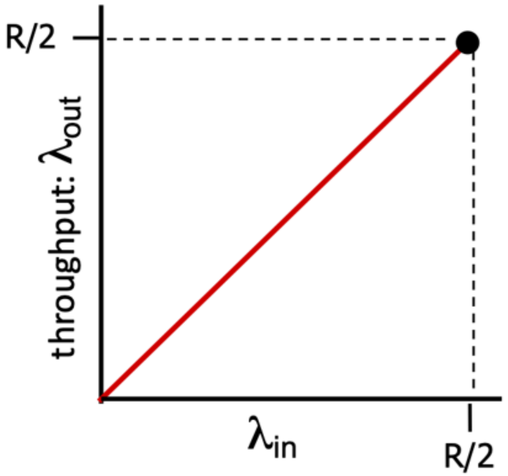
- one router, *finite* buffers
  - sender retransmits lost, timed-out packet
    - application-layer input = application-layer output: $\lambda_{in} = \lambda_{out}$
    - transport-layer input includes *retransmissions* : $\lambda'_{in} \geq \lambda_{in}$

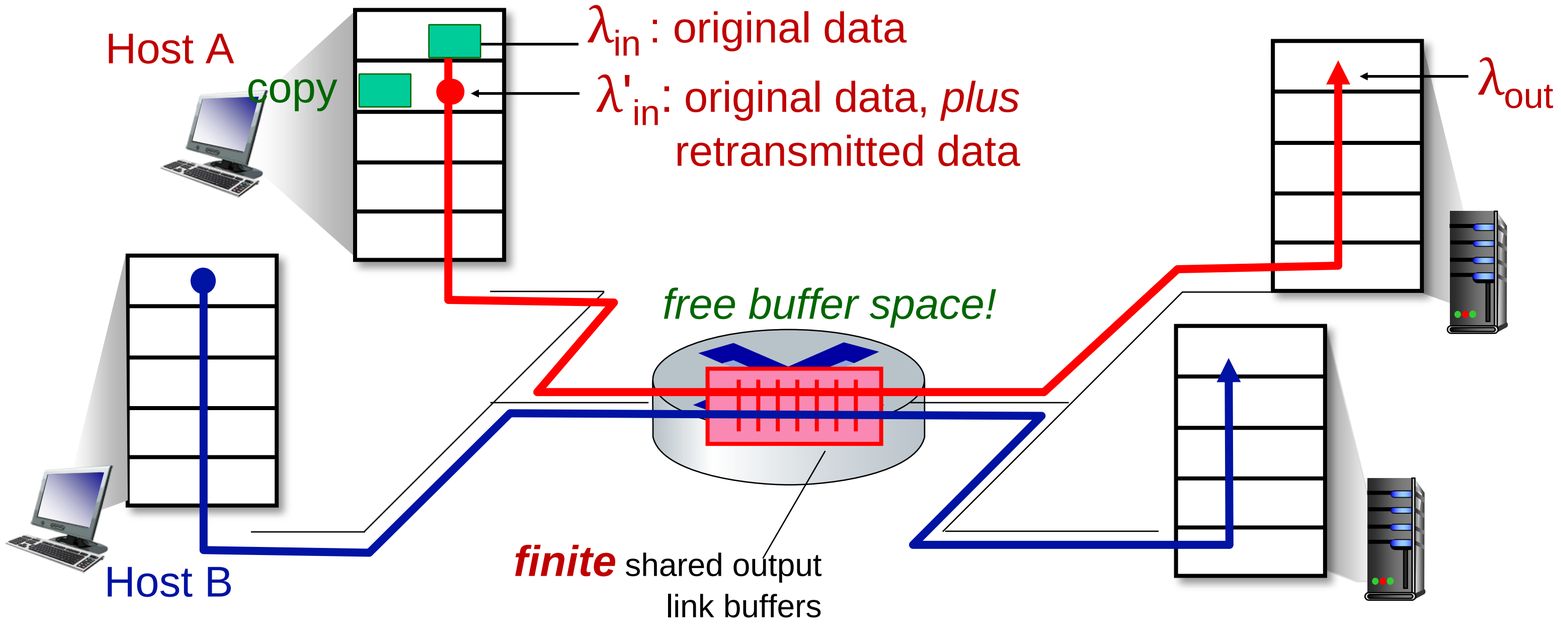


Causes/costs of congestion: scenario 2

Idealization: perfect knowledge

- sender sends only when router buffers available

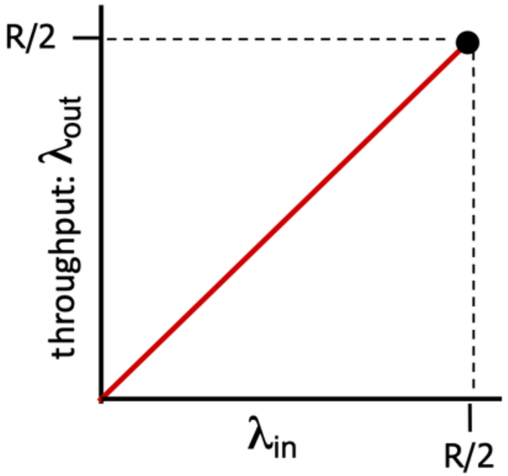


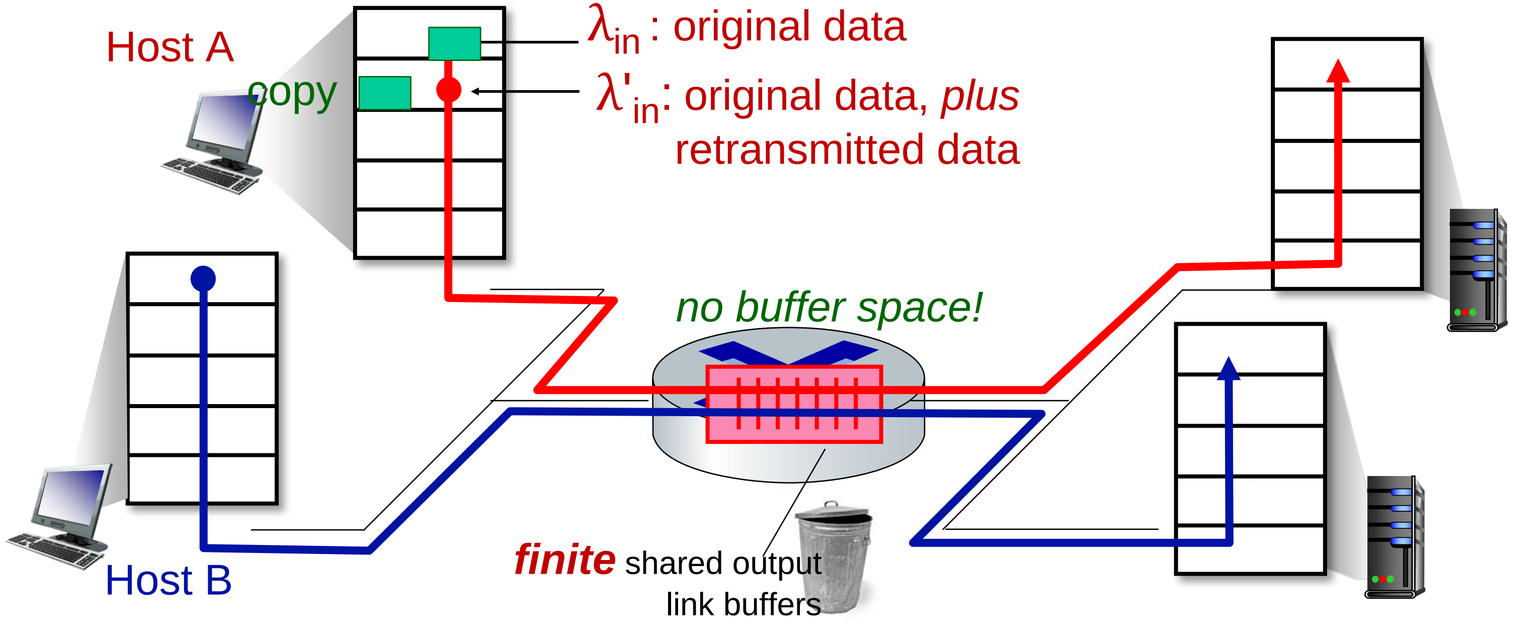


Causes/costs of congestion: scenario 2

Idealization: *some* perfect knowledge

- packets can be lost (dropped at router) due to full buffers
- sender knows when packet has been dropped: only resends if packet *known* to be lost

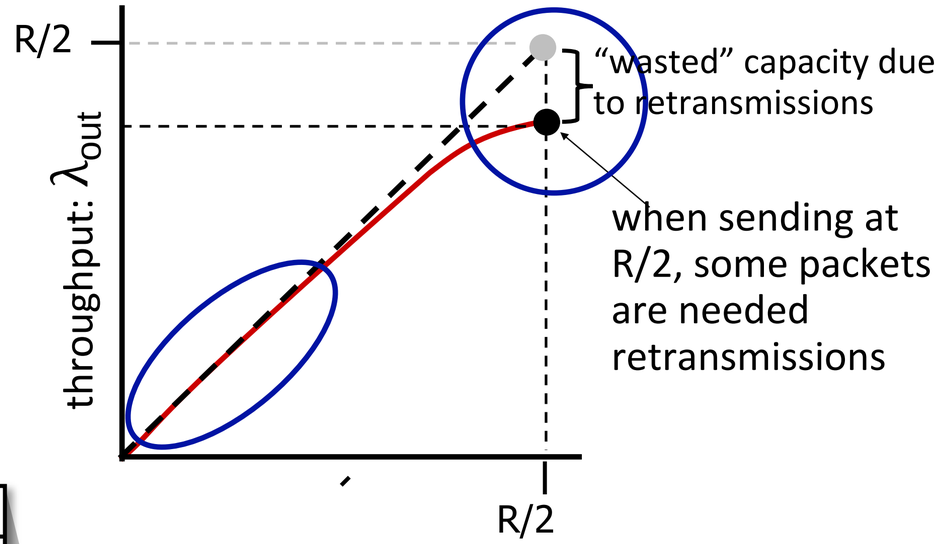


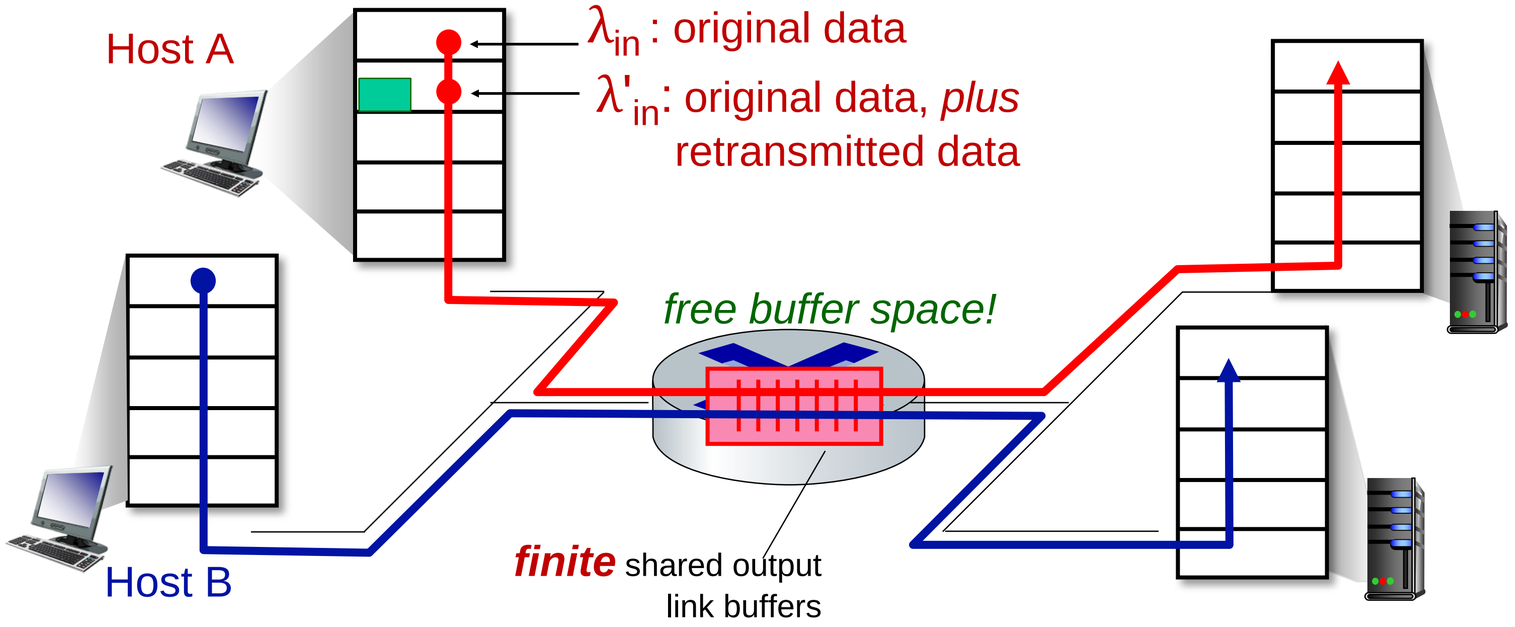


Causes/costs of congestion: scenario 2

Idealization: *some* perfect knowledge

- packets can be lost (dropped at router) due to full buffers
- sender knows when packet has been dropped: only resends if packet *known* to be lost

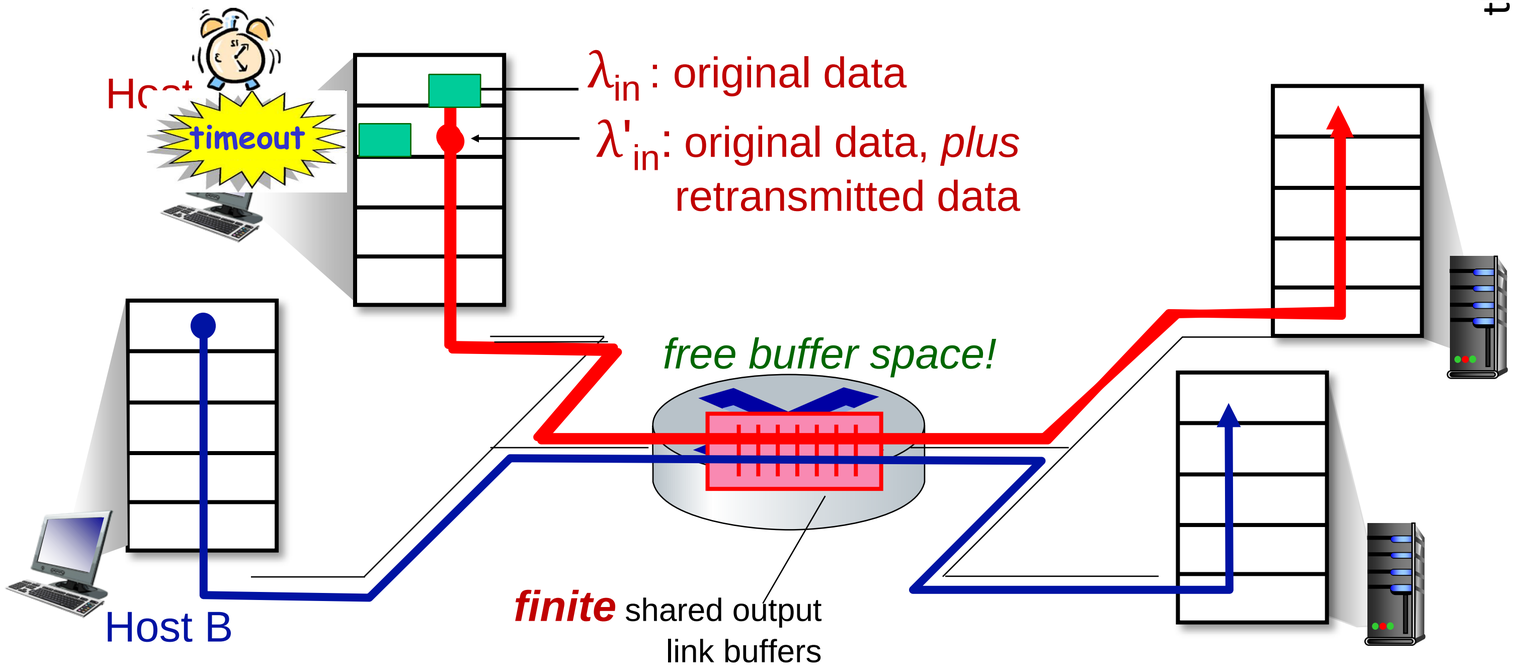


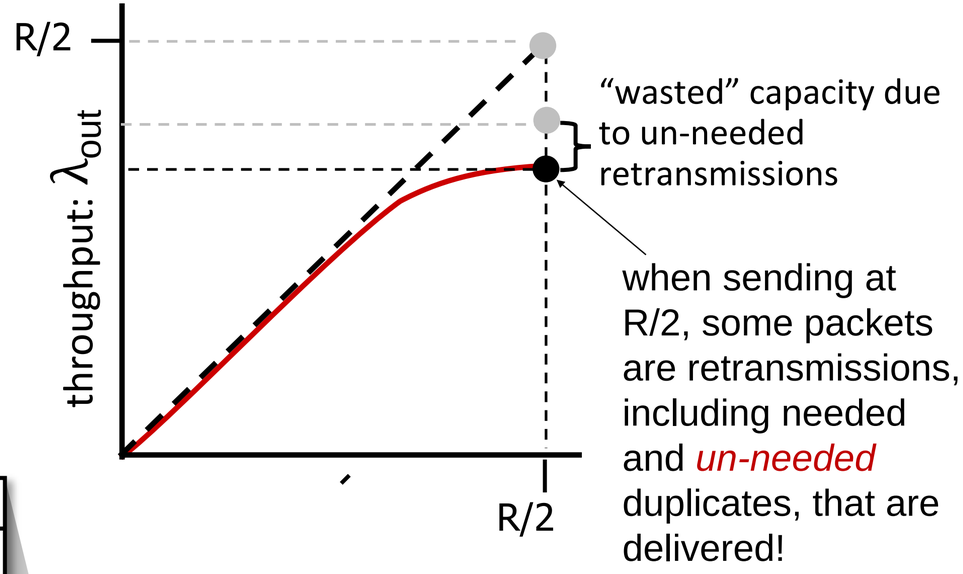


Causes/costs of congestion: scenario 2

Realistic scenario: *un-needed duplicates*

- packets can be lost, dropped at router due to full buffers – requiring retransmissions
- but sender times can time out prematurely, sending *two* copies, *both* of which are delivered

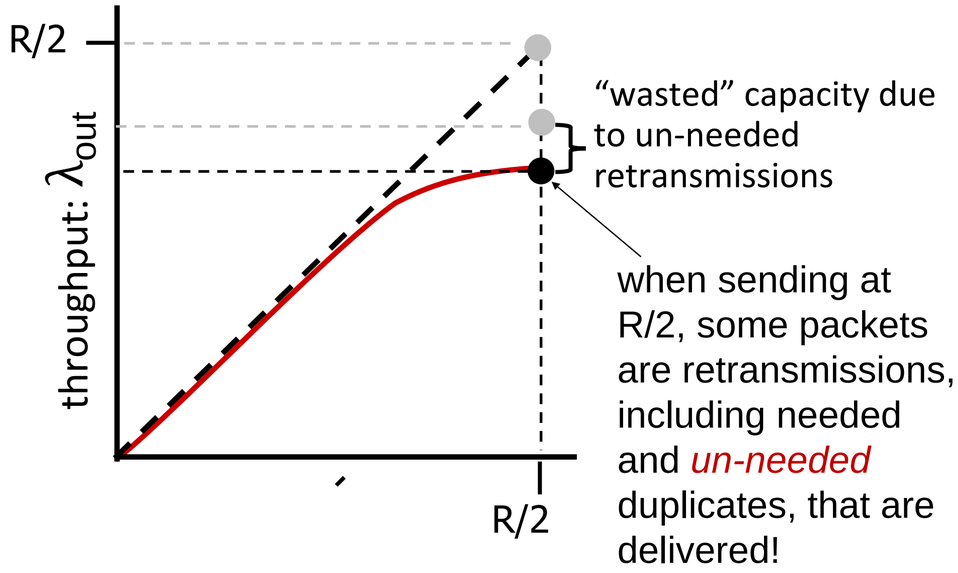




Causes/costs of congestion: scenario 2

Realistic scenario: *un-needed duplicates*

- packets can be lost, dropped at router due to full buffers – requiring retransmissions
- but sender times can time out prematurely, sending *two* copies, *both* of which are delivered



"costs" of congestion:

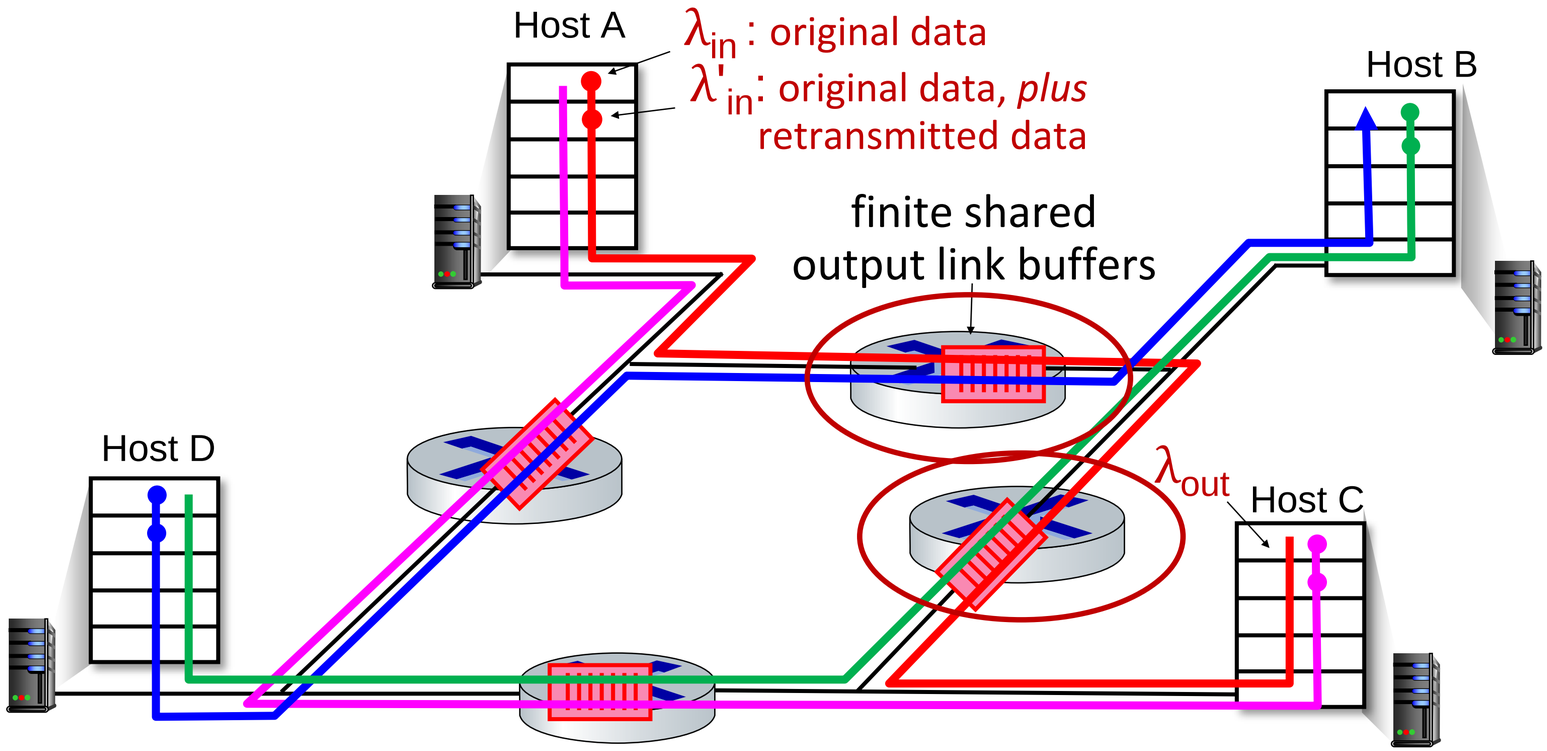
- more work (retransmission) for given receiver throughput
- unneeded retransmissions: link carries multiple copies of a packet
  - decreasing maximum achievable throughput

Causes/costs of congestion: scenario 3

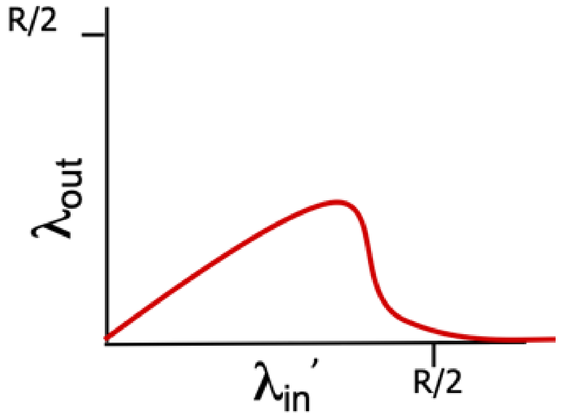
- *four* senders
- *multi-hop* paths
- timeout/retransmit

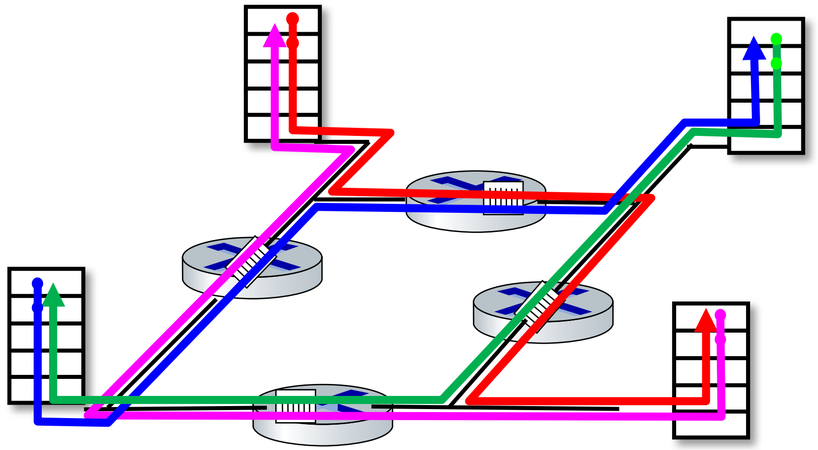
Q: what happens as λ_{in} and λ_{in}' increase ?

A: as red λ_{in}' increases, all arriving blue pkts at upper queue are dropped, blue throughput → 0



Causes/costs of congestion: scenario 3



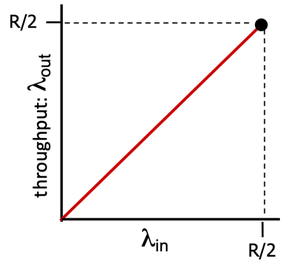


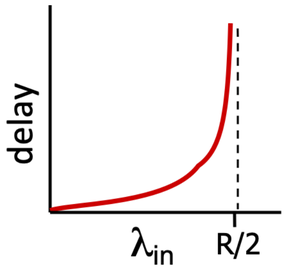
another “cost” of congestion:

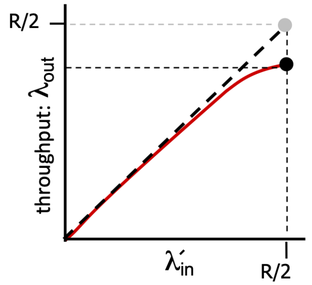
- when packet dropped, any upstream transmission capacity and buffering used for that packet was wasted!

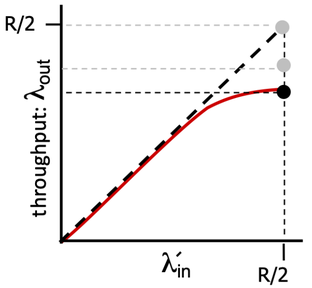
Causes/costs of congestion: insights

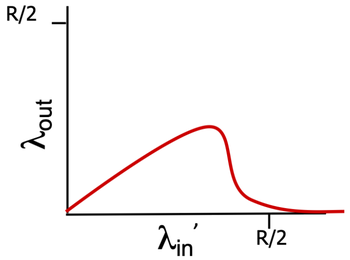
- throughput can never exceed capacity
- delay increases as capacity approached
- loss/retransmission decreases effective throughput
- un-needed duplicates further decreases effective throughput
- upstream transmission capacity / buffering wasted for packets lost downstream







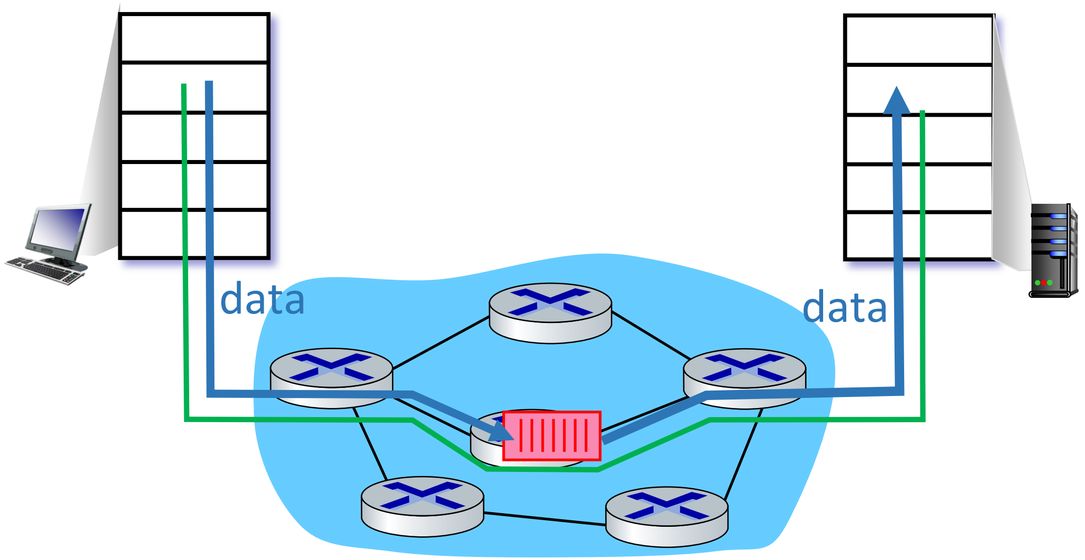




Approaches towards congestion control

End-end congestion control:

- no explicit feedback from network
- congestion *inferred* from observed loss, delay



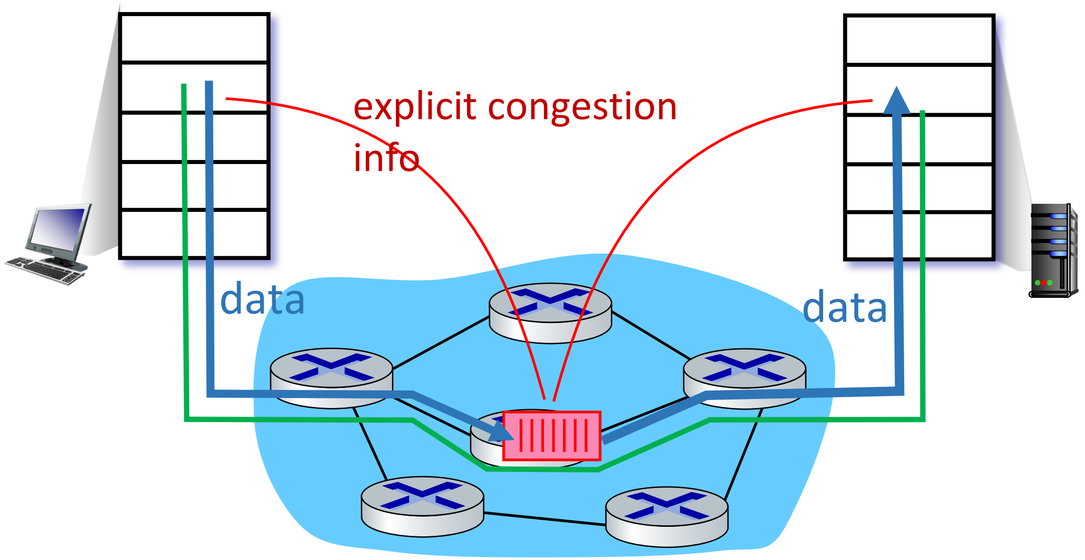
  - approach taken by TCP

Approaches towards congestion control

- TCP ECN, ATM, DECbit protocols

Network-assisted congestion control:

- routers provide *direct* feedback to sending/receiving hosts with flows passing through congested router
- may indicate congestion level or explicitly set sending rate



Outline

- Introduction
- Principles of congestion
- TCP Congestion control
- Conclusions

TCP congestion control: AIMD

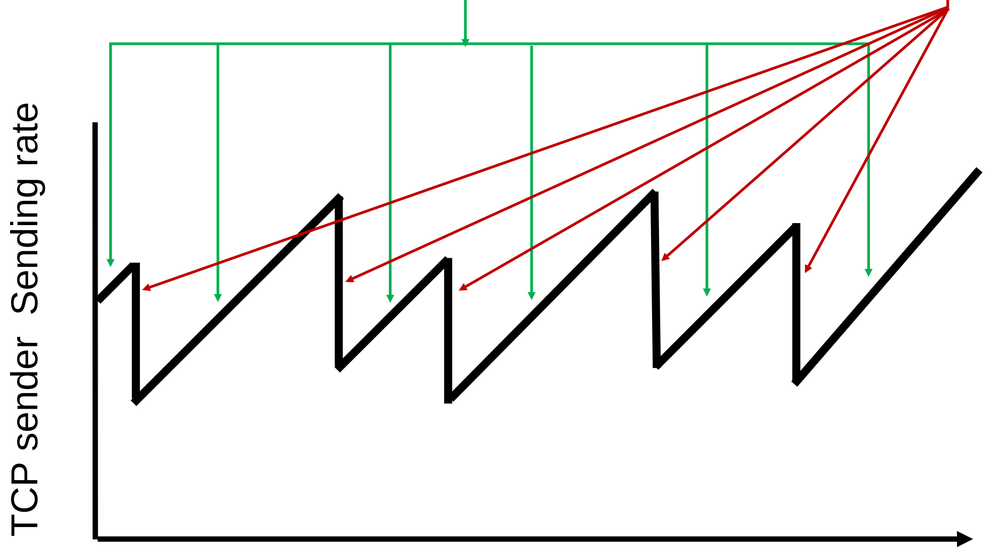
- *approach:* senders can increase sending rate until packet loss (congestion) occurs, then decrease sending rate on loss event

Additive Increase

increase sending rate by 1 maximum segment size every RTT until loss detected

Multiplicative Decrease

cut sending rate in half at each loss event



AIMD sawtooth behavior: *probing* for bandwidth

TCP AIMD: more

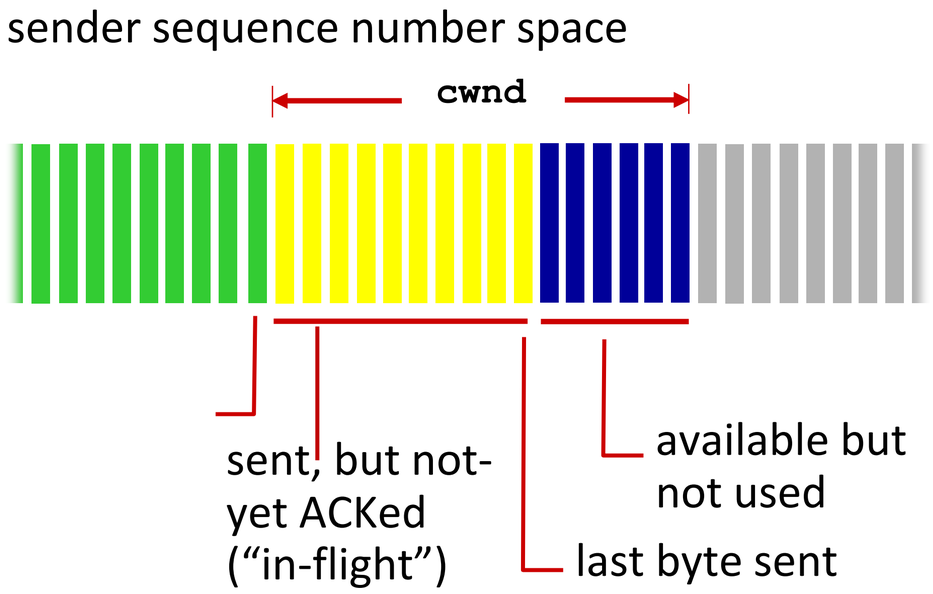
Multiplicative decrease detail: sending rate is

- Cut in half on loss detected by triple duplicate ACK (TCP Reno)
- Cut to 1 MSS (maximum segment size) when loss detected by timeout (TCP Tahoe)

Why AIMD?

- AIMD – a distributed, asynchronous algorithm – has been shown to:
  - optimize congested flow rates network wide!
  - have desirable stability properties

TCP congestion control: details



TCP sending behavior:

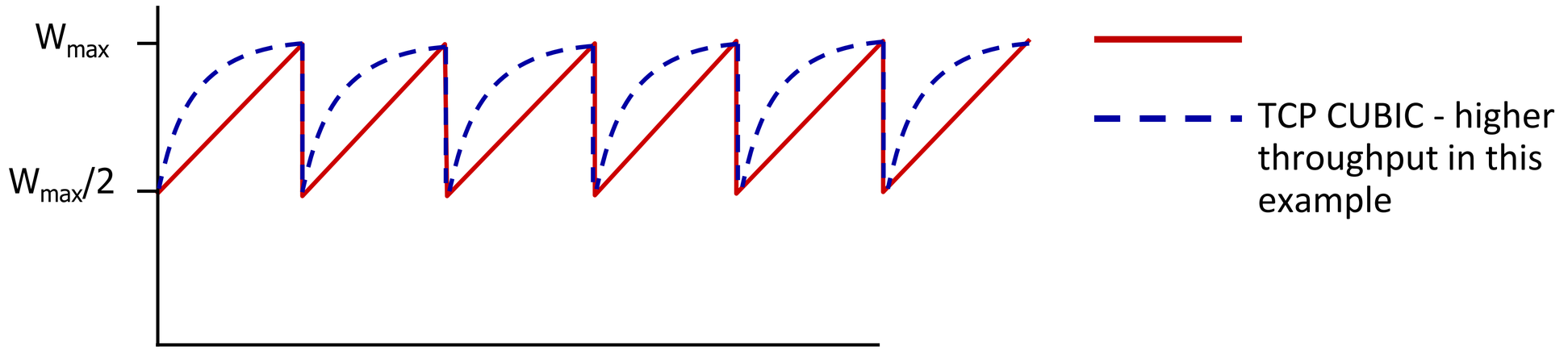
- *roughly:* send `cwnd` bytes, wait RTT for ACKS, then send more bytes

$$\text{TCP rate} \approx \frac{\texttt{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

- TCP sender limits transmission: **`LastByteSent- LastByteAcked ≤ cwnd`**
- `cwnd` is dynamically adjusted in response to observed network congestion (implementing TCP congestion control)

TCP CUBIC

- Is there a better way than AIMD to “probe” for usable bandwidth?
- Insight/intuition:
  - W_{max}: sending rate at which congestion loss was detected
  - congestion state of bottleneck link probably (?) hasn’t changed much
  - after cutting rate/window in half on loss, initially ramp to to W_{max} *faster*, but then approach W_{max} more *slowly*

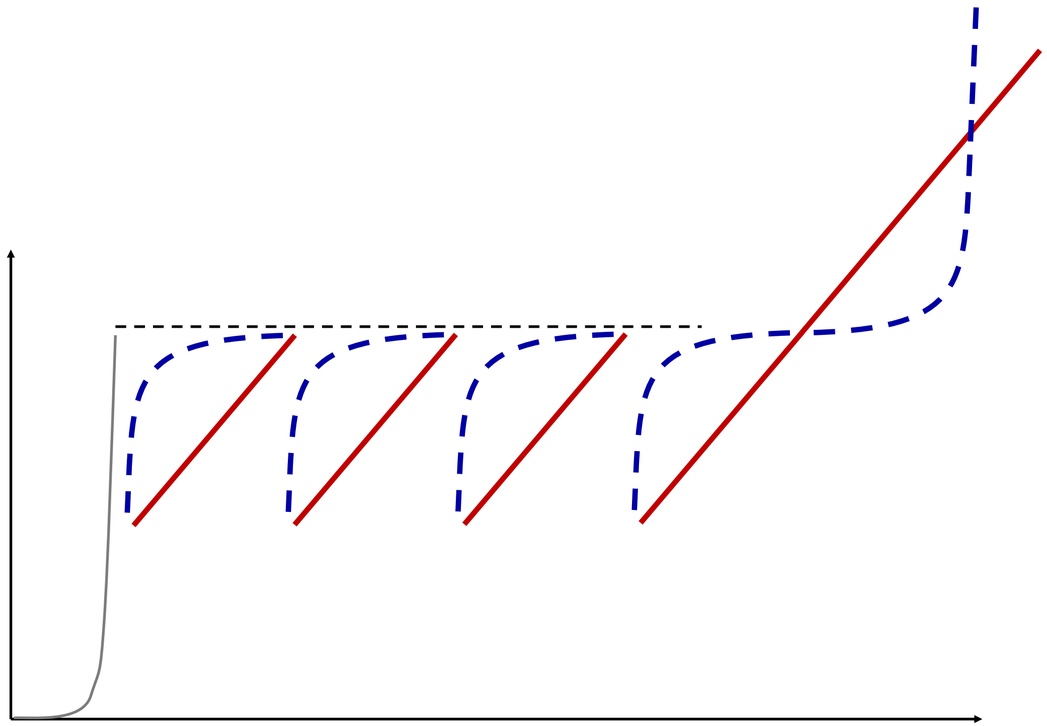


TCP CUBIC

- K: point in time when TCP window size will reach W_{max}
  - K itself is tuneable

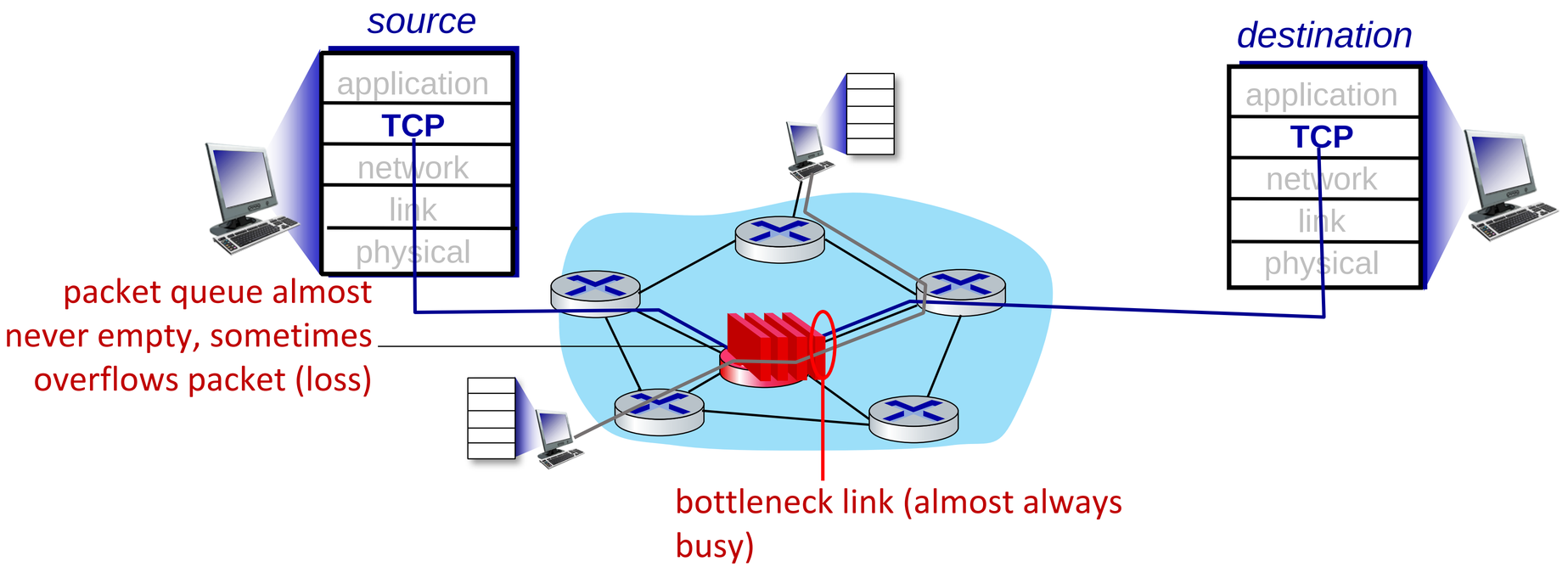
- increase W as a function of the *cube* of the distance between current time and K
  - larger increases when further away from K
  - smaller increases (cautious) when nearer K

- TCP CUBIC default in Linux, most popular TCP for popular Web servers



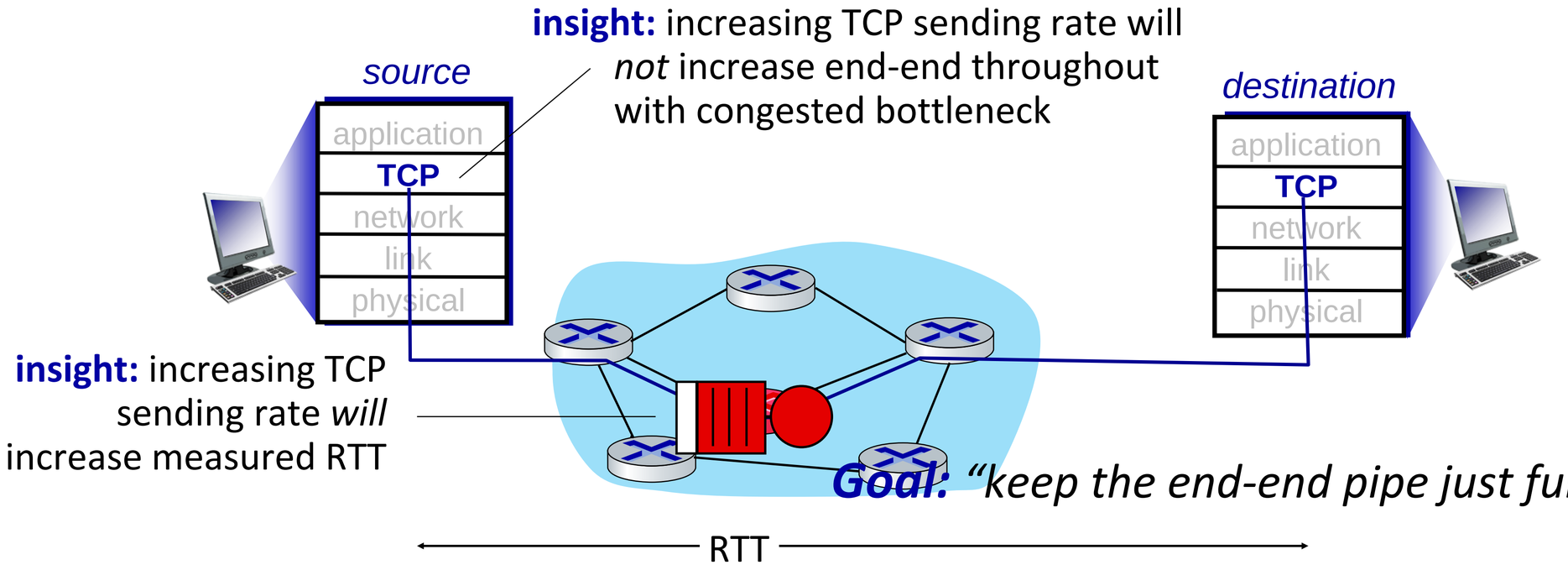
TCP and the congested “bottleneck link”

- TCP (classic, CUBIC) increase TCP’s sending rate until packet loss occurs at some router’s output: the *bottleneck link*



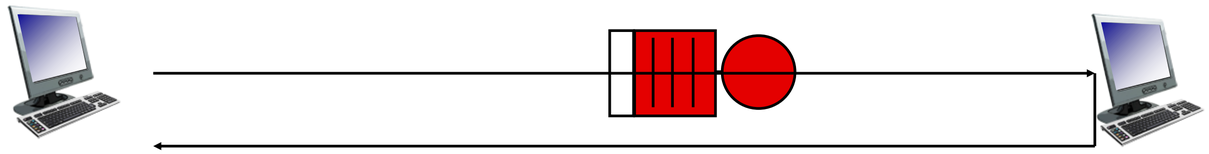
TCP and the congested “bottleneck link”

- TCP (classic, CUBIC) increase TCP’s sending rate until packet loss occurs at some router’s output: the *bottleneck link*
- understanding congestion: useful to focus on congested bottleneck link



Delay-based TCP congestion control

Keeping sender-to-receiver pipe “just full enough, but no fuller”: keep bottleneck link busy transmitting, but avoid high delays/buffering



bytes sent in
last RTT interval

Delay-based approach:

- RTT_{min} - minimum observed RTT (uncongested path)
- uncongested throughput with congestion window `cwnd` is cwnd/RTT_{min}

if measured throughput “very close” to uncongested throughput
    increase `cwnd` linearly /* since path not congested */
else if measured throughput “far below” uncongested throughout
    decrease `cwnd` linearly /* since path is congested */

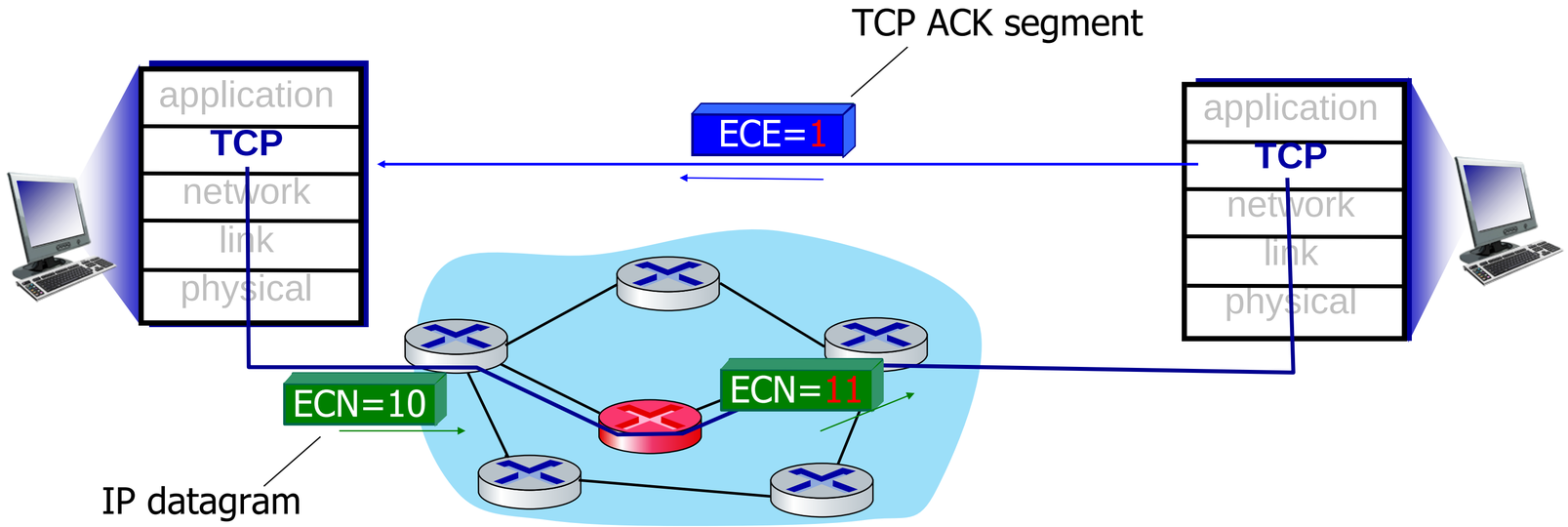
Delay-based TCP **congestion control**

- congestion control without inducing/forcing loss
- maximizing throughout (“keeping the just pipe full… ”) while keeping delay low (“…but not fuller”)
- a number of deployed TCPs take a delay-based approach
  - BBR deployed on Google’s (internal) backbone network

Explicit congestion notification (ECN)

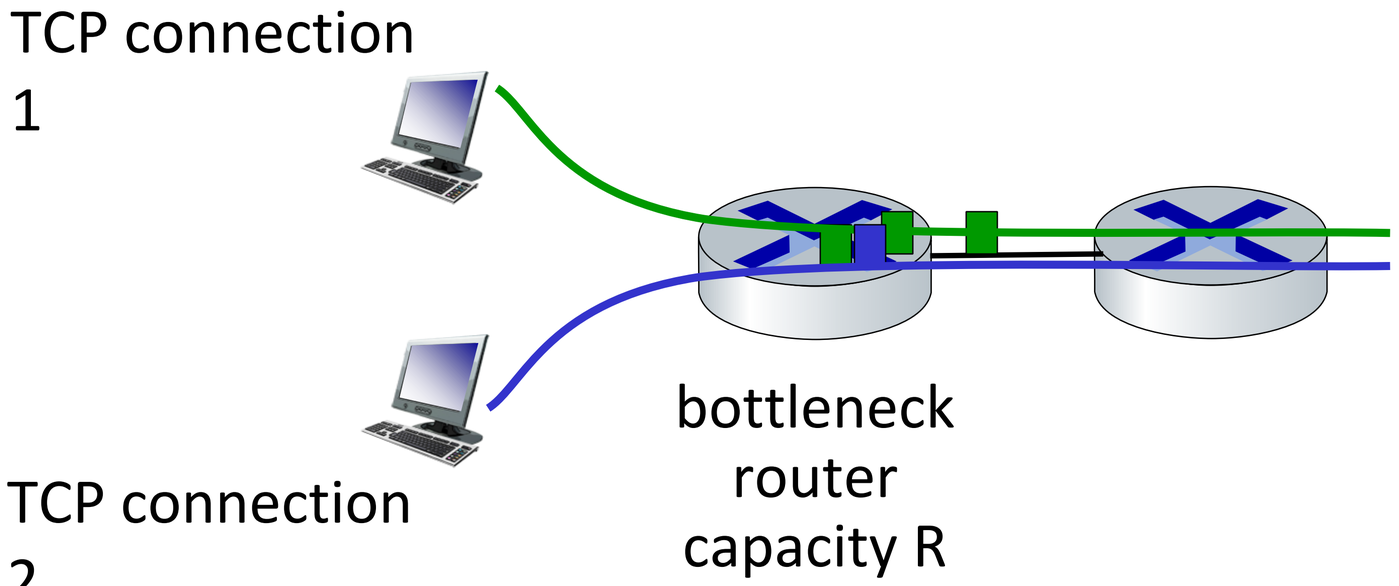
TCP deployments often implement *network-assisted* congestion control:

- two bits in IP header (ToS field) marked *by network router* to indicate congestion
  - *policy* to determine marking chosen by network operator
- congestion indication carried to destination
- destination sets ECE bit on ACK segment to notify sender of congestion
- involves both IP (IP header ECN bit marking) and TCP (TCP header C,E bit marking)



TCP fairness

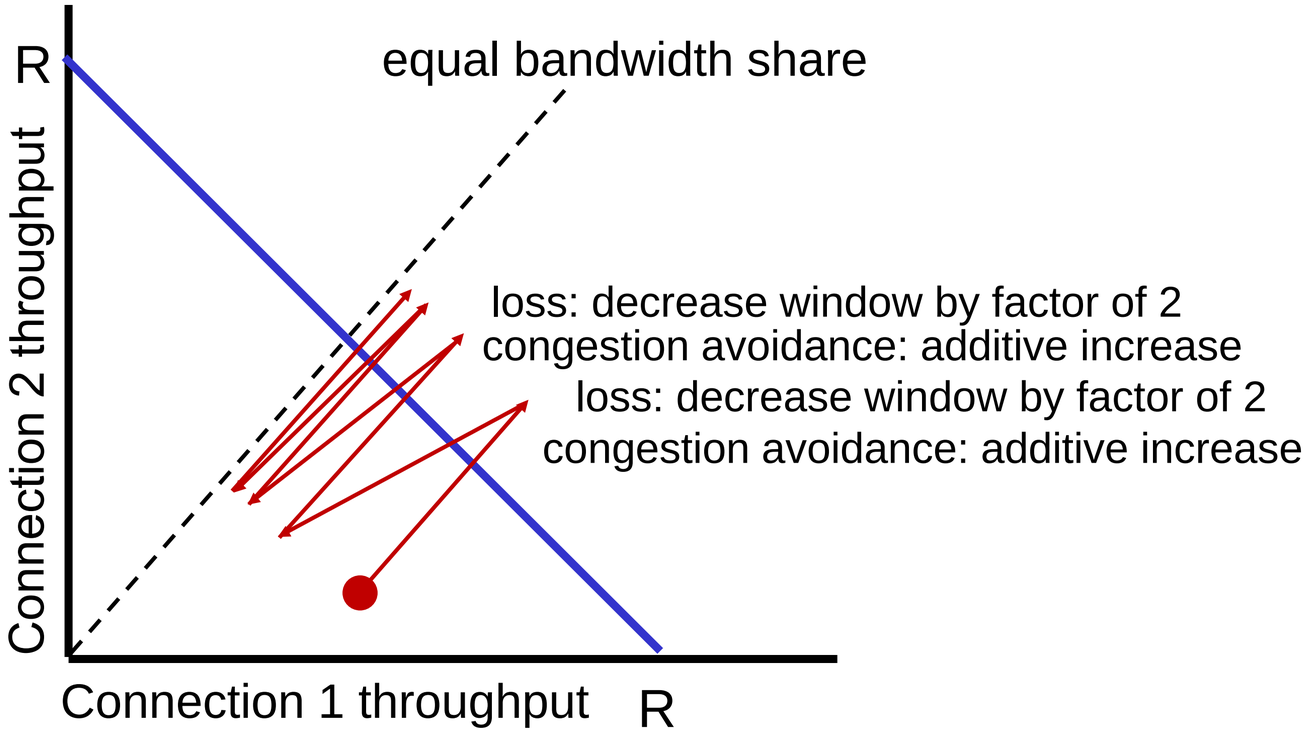
Fairness goal: if *K* TCP sessions share same bottleneck link of bandwidth *R*, each should have average rate of *R/K*



Q: is TCP Fair?

Example: two competing TCP sessions:

- additive increase gives slope of 1, as throughout increases
- multiplicative decrease decreases throughput proportionally



Is TCP fair?

A: Yes, under idealized assumptions:

- same RTT
- fixed number of sessions only in congestion avoidance

Fairness: must all network apps be “fair”?

Fairness and UDP

- multimedia apps often do not use TCP
  - do not want rate throttled by congestion control
- instead use UDP:
  - send audio/video at constant rate, tolerate packet loss
- there is no “Internet police” policing use of congestion control

Fairness, parallel TCP connections

- application can open *multiple* parallel connections between two hosts
- web browsers do this , e.g., link of rate R with 9 existing connections:
  - new app asks for 1 TCP, gets rate R/10
  - new app asks for 11 TCPs, gets R/2

Elephants Sharing the Highway: Studying TCP Fairness in Large Transfers over High Throughput Links

https://dl.acm.org/doi/fullHtml/10.1145/3624062.3624594

https://www.youtube.com/watch?v=rib_ujnMqcs&t=597s

Outline

Introduction

Principles of congestion

TCP Congestion control

Conclusions

Conclusions

> Reinventa el mundo <

- We defined the requirements for TCP windowing and buffering.
- We introduced congestion scenarios and TCP congestion control mechanisms.
- We analyzed the fairness of the TCP protocol.



UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA
TRANSFORMATEC



8.Transport Layer: TCP Congestion Control

Computer Networks



CS4054 - 2026II
PROF.: CWILLIAMS@UTEC.EDU.PE, JGONZALEZ@UTEC.EDU.PE
SRC: HARRIS & HARRIS - DIGITAL DESIGN AND COMPUTER ARCHITECTURE RISC-V